

AUTOMATION CONTROL MANUAL

ANSIBLE

PLAYBOOK ENGINEERING

Data, Logic and Organization — **Vol 2**

Full chapter detail — rendered from content Markdown

VOL 1 – FOUNDATIONS

VOL 2 – ENGINEERING
this book

VOL 3 – PRODUCTION

TABLE OF CONTENTS

06	Variables · Facts · Registers	03	08	Templates and Handlers	14
07	Conditionals · Loops · Filters	08	09	Roles and Collections	18
			A	Cheatsheet	22

[HOW TO READ]

Chapters render the full content Markdown: data flow, branching, files, organization, labs, interview checks.

Version anchor: Same pins as Vol 1 (core 2.16+, Python 3.11+); extra-vars outranks every file..

Data lane – what varies per host



VARIABLES · FACTS · REGISTERS

▶ WHAT – Three sources of data

Variables parameterize desired state. Facts are auto-gathered system data. Registers capture task output for later use.



- **Variables:** author-defined values (app_port: 8080) from inventory, group_vars/, host_vars/, play, task, or --extra-vars.
- **Facts:** discovered values (ansible_os_family, ansible_default_ipv4.address) collected by ansible.builtin.setup at play start (gather_facts).
- **Registers:** per-host results of a prior task (register: host_out), used by later tasks, conditionals, or templates.

▶ WHY – One playbook, many hosts

Same playbook must behave differently per host: web-01 listens on 8080 with one certificate, web-02 on 8081 with another. Hard-coding values per host duplicates playbooks. Data flow explains *how* values travel from inventory and discovery into modules, templates, and conditionals.

▶ WHERE DOES IT FIT – Data lane

```
Inventory vars → group_vars → host_vars → play vars → task vars → facts/registers →
module/template/conditional
```

This lane sits after mechanics (PDF1): playbooks declare *what*, variables decide *with which values per target*. Logic (Ch 07) and templates (Ch 08) consume this data.

VARIABLES · FACTS · REGISTERS

▶ HOW IT WORKS – Precedence, facts, registers

Precedence exists because specificity must win: general defaults lose to host-specific values. Simplified order (low → high):

```
role defaults → group_vars/all → group_vars/web → host_vars/web-01 → play vars → task vars → extra-
vars
```

Collision example: `group_vars/all.yml` sets `app_port: 8080`, `host_vars/web-01.yml` sets `app_port: 9090` → `web-01` renders 9090, others render 8080. `ansible-playbook -e app_port=9999` overrides everything (operator intent wins for one run).

Facts: `gather_facts: true` (default) runs setup first, populating `ansible_facts`. Disable with `gather_facts: false` for speed when facts are unused. Registers: `register` stores stdout, stderr, rc, changed per host — never shared across hosts unless `hostvars` is read explicitly.

ASPECT	VARIABLE	FACT	REGISTER
Author	You write it	Ansible discovers it	A task captures it
Example	<code>app_port: 8080</code>	<code>ansible_facts.os_family</code>	<code>host_out.stdout</code>
Refresh	File-defined rank	Re-gathered every run	Re-registered every run
Override	<code>--extra-vars</code> wins	Cannot override, only use	Read via <code>hostvars</code> cross-host

VARIABLES · FACTS · REGISTERS

▶ PRACTICAL – Observe data flow

```
- name: Show data flow
  hosts: web
  vars:
    app_port: 8080
  tasks:
    - name: Show variable
      ansible.builtin.debug:
        msg: "port {{ app_port }}"

    - name: Capture hostname
      ansible.builtin.command: cat /etc/hostname
      register: host_out
      changed_when: false

    - name: Show captured output
      ansible.builtin.debug:
        var: host_out.stdout

    - name: Show discovered fact
      ansible.builtin.debug:
        msg: "family {{ ansible_facts.os_family }}"
```

COMMAND

```
ansible-playbook -i inventory.ini data.yml -e app_port=9999
```

Meaning: run play with CLI override. Expected: msg: "port 9999" on all hosts — proves extra-vars beats file vars. Remove -e → web-01 shows 9090 (host_vars), others 8080.

Why it matters: proves precedence is deterministic, not magic. Common failure: quoting msg="port {{ app_port }}" with double quotes inside YAML double quotes → use single outer or block scalar.

VARIABLES • FACTS • REGISTERS

► WHAT CAN FAIL

SYMPTOM	LIKELY CAUSE	PREVENTION
AnsibleUndefinedVariable: 'app_port' is undefined	Typo or vars file in wrong directory	group_vars/ must sit beside inventory file; verify with ansible-inventory --graph --vars
Precedence surprise (wrong port)	Higher-specificity file wins silently	debug: var=app_port per host before template task
host_out.stdout empty	Task skipped/failed before register	Check host_out.rc, host_out.skipped; guard with when: host_out.rc is defined
Facts missing	gather_facts: false set globally	Re-enable per play or call ansible.builtin.setup explicitly

[GOTCHA]

Precedence is not inheritance depth — it is specificity + source rank. Group of one host still loses to that host's host_vars. Always verify with --list-hosts + debug before templating.

► REAL-WORLD CONNECTION

Team stores defaults in group_vars/all.yml (port, TLS paths), per-environment overrides in group_vars/prod.yml, per-host exceptions in host_vars/. Promotion from staging to prod changes inventory path, not playbook. changed counts stay clean because data varies, not tasks.

VARIABLES · FACTS · REGISTERS

▶ LAB – Observe / Execute / Verify

Objective: prove precedence + facts + registers. Prereqs: group_vars/all.yml (app_port: 8080), host_vars/web-01.yml (app_port: 9090).

1. `ansible-inventory -i inventory.ini --graph --vars` → Observe merged vars per host
2. `ansible-playbook -i inventory.ini data.yml --check` → Execute dry run, watch msg per host
3. Verify web-01 shows 9090, others 8080. Explain: host specificity wins.
4. Re-run with `-e app_port=9999` → Verify all show 9999. Explain: operator override is highest rank.

Failure drill: rename host_vars/ to hostvars/ → re-run → undefined/wrong value → fix directory name.

▶ Interview check

Interview Where do variables come from, and which wins?

Inventory, group/host vars files, play/task vars, facts, registers, extra-vars. More specific beats more general; extra-vars beats files. Facts are discovered, registers are captured per host from prior task results.

[NEXT]

Data varies per host. Next: branch and repeat with that data — conditionals, loops, filters.

CONDITIONALS • LOOPS • FILTERS

▶ WHAT – Branch, repeat, transform

Conditionals (when) branch tasks. Loops (loop) repeat one task over many items. Filters transform data inside Jinja2 expressions (`{{ value | filter }}`).

```
- name: Install nginx only on web
  ansible.builtin.package:
    name: nginx
    state: present
  when: inventory_hostname in groups['web']

- name: Ensure users exist
  ansible.builtin.user:
    name: "{{ item }}"
    state: present
  loop: [alice, bob]
```

Jinja2 filters: `{{ app_port | default(8080) }}`, `{{ groups['web'] | length }}`, `{{ app_name | upper }}`.

▶ WHY – Fleets diverge

Real fleets are not uniform: web needs nginx, db needs postgres; ten users, not one. Without logic the author copies tasks per host — drift follows. when keeps one task with a guard; loop keeps one task with many items; filters keep expressions safe when data is missing.

▶ WHERE DOES IT FIT – Logic consumes data

Variables/facts/registers (Ch 06) → when/loop/filter (this chapter) → template/handler (Ch 08)

Conditionals read facts and registers (`ansible_facts.os_family == "Debian"`, `host_out.rc == 0`). Loops feed `{{ item }}` into modules. Filters guard undefined data (`| default`) so a missing var fails gracefully instead of aborting the play.

CONDITIONALS · LOOPS · FILTERS

▶ HOW IT WORKS – Evaluation per host

when is a Jinja2 expression evaluated per host, without `{{ }}` delimiters. loop executes the task once per item, with `item` and `ansible_loop` metadata. Filters pipe left value through right function; they chain (`{{ name | lower | trim }}`).

Combined example:

```
- name: Deploy app config per host
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/app/app.conf
  loop: "{{ app_instances }}"
  when: app_enabled | default(true)
```

Loop over `app_instances` (from group vars), skipped entirely when `app_enabled` is false. `loop_control: label` keeps output readable for long lists.

BRANCH PER HOST AT RUN TIME

WHEN TRUE

Task runs on this host → module converges → result recorded

WHEN FALSE

Task SKIPPED on this host → play continues → no change, no failure

CONDITIONALS · LOOPS · FILTERS

▶ FILTER FIELD GUIDE – The dozen that carry you

Filters group by job. Learn these twelve and you can read any playbook:

JOB	FILTERS	EXAMPLE
Missing-safe	default, is defined	<code>{{ app_port default(8080) }}</code>
Branch in one line	ternary	<code>{{ tls_enabled ternary('https', 'http') }}</code>
Types	int, bool, string, list	<code>{{ replicas int + 1 }}</code>
Lists	length, join, map, select, unique, sort	<code>{{ groups['web'] length }}</code> , <code>{{ users map(attribute='name') join(',') }}</code>
Dicts	dict2items, combine	<code>{% for kv in app_env dict2items %} with kv.key / kv.value</code>
Text	upper, lower, trim, replace	<code>{{ app_name lower trim }}</code>
Data formats	to_json, from_yaml, to_nice_yaml	<code>{{ cfg to_nice_yaml }}</code> for debug output

The safety pattern first: when: myvar is defined and myvar == "x" never explodes on missing hosts; `{{ myvar | default('x') }}` never aborts a template. Write every optional value one of these two ways.

CONDITIONALS • LOOPS • FILTERS

▶ LOOP VARIANTS – Beyond flat lists

Dictionaries need dict2items (a dict cannot loop directly):

```
- name: Export environment entries
  ansible.builtin.lineinfile:
    path: /etc/app/env
    line: "{{ item.key }}={{ item.value }}"
    loop: "{{ app_env | dict2items }}"
```

Long or slow loops stay readable and controllable:

```
- name: Pull service images
  ansible.builtin.command: docker pull {{ item }}
  loop: "{{ images }}"
  loop_control:
    label: "{{ item }}"
    pause: 2
```

Polling waits for reality instead of sleeping blindly:

```
- name: Wait for healthy endpoint
  ansible.builtin.uri:
    url: "http://{{ inventory_hostname }}:8080/health"
  register: health
  until: health.status == 200
  retries: 30
  delay: 10
```

until re-runs the task until the condition holds (or retries run out); combine with changed_when: false for pure checks so successful polls report OK, not CHANGED.

CONDITIONALS · LOOPS · FILTERS

▶ PRACTICAL – Run branch + repeat

COMMAND

```
ansible web -i inventory.ini -m ansible.builtin.debug -a "msg={{ ansible_facts.os_family }}"
```

Meaning: show discovered family per host. Expected: Debian / RedHat per target — proves facts exist before when uses them.

```
- name: Guarded install
  hosts: all
  tasks:
    - name: Install nginx on Debian web only
      ansible.builtin.package:
        name: nginx
        state: present
      when:
        - inventory_hostname in groups['web']
        - ansible_facts.os_family == "Debian"
```

Why it matters: two-condition guard prevents installing on wrong OS family. Common failure: when: "{{ condition }}" with delimiters → always truthy string; write bare when: condition.

CONDITIONALS • LOOPS • FILTERS

► WHAT CAN FAIL

SYMPTOM	LIKELY CAUSE	PREVENTION
template error while templating string	Undefined var or bad filter chain	<code>{{ var default('x') }}</code> , test with debug first
Task skipped unexpectedly	when evaluated false on that host	<code>--check -vv</code> shows skip reason per host
when with <code>{{ }}</code> always runs	Non-empty string is truthy	Bare expression: <code>when: rc == 0</code>
Loop output unreadable (100 items)	Default item echo	<code>loop_control: { label: "{{ item.name }}" }</code>

[GOTCHA]

when does not use `{{ }}`. `when: ansible_facts.os_family == "Debian" converges`. `when: "{{ ... }}"` is a non-empty string — always true.

► REAL-WORLD CONNECTION

Patch window: same playbook against staging then prod. `when: env == "prod"` gates the disruptive restart; loop over `critical_services` restarts each; | default guards hosts missing the optional `maintenance_window` var. `--limit web` plus `--check --diff` previews exactly which hosts branch where.

► LAB – Observe / Execute / Verify

Objective: prove branch + repeat. Prereqs: inventory with `web` and `db` groups.

1. `ansible all -i inventory.ini -m ansible.builtin.setup -a "filter=ansible_os_family" → Observe facts`
2. Run guarded-install play with `--check --diff → Execute`, watch per-host skipped vs changed
3. Verify `db` hosts skip, `web` Debian hosts change. Explain: guard evaluated per host.
4. Add loop: `[alice, bob]` user task → re-run → Verify two ok items per host.

Failure drill: set `when: "{{ false }}" → task runs anyway → fix to bare when: false.`

► Interview check

Interview when vs loop vs filter — what does each do?

when decides per host whether the task runs. Loop repeats one task per item with `item` bound. Filters transform values inside `{{ }}` (defaults, case, length) so expressions survive missing or messy data.

[NEXT]

Logic branches and repeats. Next: render per-host files from that data and restart services once — templates + handlers.

TEMPLATES AND HANDLERS

► WHAT – Render files, defer restarts

`ansible.builtin.template` renders a Jinja2 file (`src: *.j2`) to the managed node. Handlers run once at end of play — only if notified by a task that reported `CHANGED`.



Five notifying tasks still restart the service exactly once, at play end.

```

- name: Render nginx config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    mode: '0644'
  notify: Restart nginx

handlers:
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
  
```

Template (`nginx.conf.j2`):

```

server {
  listen {{ app_port | default(8080) }};
  server_name {{ inventory_hostname }};
}
  
```

► WHY – Config varies, restarts are expensive

Config files differ per host (port, TLS path, server name) — `copy` ships identical bytes, `template` ships per-host rendering. Restarting a service per task is noisy and can flap: five config tasks → five restarts. Handlers batch: five tasks notify, handler runs once at end, only if something actually changed.

TEMPLATES AND HANDLERS

▶ WHERE DOES IT FIT – Data + logic → files + actions

Variables/facts (Ch 06) + when/loop (Ch 07) → template renders → compare → CHANGED → notify queue → handler once

copy = static bytes. template = bytes computed from data. lineinfile/blockinfile = surgical edits (prefer full template when the file is owned; prefer line edits for foreign files like /etc/sudoers.d/ fragments).

▶ HOW IT WORKS – Compare, queue, flush

Template renders locally, compares checksum with remote dest, transfers only on difference → CHANGED or OK. notify appends handler name to per-play queue; duplicates collapse to one entry. At play end (or meta: flush_handlers mid-play), each queued handler runs once in handler order, not notify order. listen: groups multiple notify names onto one handler.

```
handlers:
- name: Restart web stack
  listen: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

validate: guards syntax before install (validate: "nginx -t -c %s"); failed validation → task FAILED, dest untouched.

TEMPLATES AND HANDLERS

► PRACTICAL – Observe notify-on-change

COMMAND

```
ansible-playbook -i inventory.ini site.yml
```

First run: TASK [Render nginx config] changed: [web-01] → RUNNING HANDLER [Restart nginx] changed: [web-01]. Second run: both ok — handler silent. Proof of deferred idempotent restart.

Why it matters: handler silence on second run is the convergence signal for config management. Common failure: handler name typo → ERROR! The requested handler 'Restart ngnix' was not found — names match exactly unless listen aliases.

► WHAT CAN FAIL

SYMPTOM	LIKELY CAUSE	PREVENTION
could not find src: nginx.conf.j2	Wrong src path (role vs playbook relative)	src relative to templates/ in role, or playbook dir; verify with --check -vv
Handler never runs	Task reports OK (already converged) or name mismatch	Change a var to force CHANGED; use listen: for aliases
Handler runs every time	Template renders nondeterministically (timestamp inside)	Remove volatile content from template; use validate + --diff to inspect
Bad config deployed	No validate	validate: "nginx -t -c %s" on template task

[GOTCHA]

"OK" on template means handler correctly stayed silent. Do not force handlers with --force-handlers in normal runs — that flag is for failure analysis, not convergence.

► REAL-WORLD CONNECTION

TLS rotation: new cert vars in group_vars/prod.yml → template re-renders per host → only changed hosts notify restart → rolling update with serial: 1 restarts one host at a time. --diff in review shows exactly which lines change per host before approval.

TEMPLATES AND HANDLERS

▶ LAB – Observe / Execute / Verify

Objective: prove render → notify → once. Prereqs: `nginx.conf.j2` with `{{ app_port }}`.

1. Run play → Observe changed + handler run
2. Re-run → Verify ok, handler silent. Explain: no diff, no notify.
3. Change `app_port` in `host_vars/web-01.yml` → run → Verify only `web-01` changes + handler runs for it.
4. Insert meta: `flush_handlers` between two config tasks → Verify handler runs mid-play.

Failure drill: rename handler → run → not-found error → fix with `listen: alias`.

▶ Interview check

Interview Why handlers instead of restarting in the task?

Tasks may repeat; restarts must not. Handlers collapse many notifications into one run at play end, and only when a notifying task actually changed something — batching + convergence in one mechanism.

[NEXT]

Single play grows large with templates and handlers. Next: split into reusable units — roles and collections.

ROLES AND COLLECTIONS

► WHAT – Reusable units and their distribution

Role = reusable unit with conventional directories (tasks, handlers, templates, defaults, vars). Collection = versioned distribution of roles plus modules and plugins, addressed by fully qualified names (namespace.collection).

```
roles/web/ tasks/main.yml handlers/main.yml templates/nginx.conf.j2 defaults/main.yml
vars/main.yml meta/main.yml
```

Conventional layout: tunables in defaults (lose), assertions in vars (win).

```
roles/web/
tasks/main.yml
handlers/main.yml
templates/nginx.conf.j2
defaults/main.yml
vars/main.yml
meta/main.yml
```

```
- name: Configure web fleet
  hosts: web
  roles:
    - role: web
      vars: { app_port: 8080 }
```

Install: `ansible-galaxy collection install community.general.FQCN` in tasks: `community.general.ufw`, `ansible.builtin.template`.

► WHY – Playbooks do not scale flat

One `site.yml` with 200 tasks is unreviewable: variables collide, handlers duplicate, copy-paste spreads. Roles give boundaries (defaults vs vars, tasks vs handlers); collections give versioning and sharing (pin `requirements.yml`, upgrade deliberately). Galaxy and private automation hub become the package registry for automation.

ROLES AND COLLECTIONS

▶ WHERE DOES IT FIT – Organization after mechanics

```
Play → task → module (Ch 04) + data (Ch 06) + logic (Ch 07) + files (Ch 08)
→ role bundles them → collection ships them → ansible.cfg/CLI locates them
```

defaults/main.yml = lowest precedence (safe to override). vars/main.yml = high precedence (role asserts).
requirements.yml pins collections per project — same discipline as requirements.txt.

▶ HOW IT WORKS – Resolution and inclusion

Role search path: playbook roles/ dir → roles_path in ansible.cfg → Galaxy default. Task inside role resolves templates/ and files/ relative to the role. import_role is static (parsed when playbook loads — when applies per task inside). include_role is dynamic (evaluated at execution — can loop over roles).

```
- name: Static web role
  ansible.builtin.import_role:
    name: web

- name: Dynamic per-env roles
  ansible.builtin.include_role:
    name: "{{ item }}"
  loop: [base, web]
```

Collections: ansible-galaxy collection install -r requirements.yml; reference namespace.collection.module.
Short names (template, package) still resolve today but are obsolete — lint flags them, content uses FQCN throughout.
ansible.cfg loading order: ANSIBLE_CONFIG env → ./ansible.cfg → ~/.ansible.cfg → /etc/ansible/ansible.cfg.
Keep project-local ansible.cfg (roles_path, inventory, forks) committed beside site.yml.

ROLES AND COLLECTIONS

► PRACTICAL – Convert play to role

COMMAND

```
ansible-galaxy role init roles/web
ansible-playbook -i inventory.ini site.yml --check --diff
```

Meaning: scaffold conventional layout, then dry-run role-based play. Expected: same changed/ok counts as flat play — proves refactor preserved behavior.

Why it matters: refactor without behavior change is the organization proof. Common failure: couldn't resolve module/action 'my_module' → collection not installed or FQCN missing → `ansible-galaxy collection install -r requirements.yml`.

► WHAT CAN FAIL

SYMPTOM	LIKELY CAUSE	PREVENTION
couldn't resolve module/action	Collection missing or short name	FQCN + requirements.yml + ansible-galaxy install -r
Role not found	roles_path wrong or role beside wrong playbook	ansible-config dump grep -i roles_path; keep roles/ beside site.yml
import_* vs include_* surprise	Static vs dynamic timing	import for fixed structure; include for loops/conditionals on the include itself
Vars collision across roles	Same var name in two roles	Namespace vars (web_app_port), defaults for overridables

[GOTCHA]

defaults/ loses to almost everything; vars/ beats almost everything. Put tunables in defaults, assertions in vars — reversing them makes overrides silently fail.

► REAL-WORLD CONNECTION

Platform team ships `acme.web` collection v2.4. App teams pin `acme.web`: `">=2.0,<3.0"` in `requirements.yml`, override `acme_web_port` in `group_vars/`. Upgrade = bump pin, `--check --diff` in staging, promote. Roles become the API between platform and app teams.

ROLES AND COLLECTIONS

▶ LAB – Observe / Execute / Verify

Objective: extract tasks into role without behavior change. Prereqs: working flat `site.yml` from Ch 04.

1. `ansible-galaxy role init roles/web` → Observe scaffold
2. Move tasks/handlers/templates/vars into role dirs → point `site.yml` at roles: `[web]`
3. `ansible-playbook -i inventory.ini site.yml --check --diff` → Verify identical planned changes. Explain: organization, not behavior, changed.
4. Add `requirements.yml` with one collection → `ansible-galaxy collection install -r requirements.yml` → Verify FQCN task resolves.

Failure drill: break `roles_path` in `ansible.cfg` → not-found error → `ansible-config dump` → fix path.

▶ Interview check

Interview Role vs collection vs playbook? import vs include?

Playbook selects hosts and composes roles. Role bundles tasks/handlers/templates/vars for one concern. Collection versions and ships roles+modules+plugins. `import_*` resolves at parse (static), `include_*` at run (dynamic, loopable).

[NEXT]

Organized automation is ready to be secured. Next: privilege boundaries and secrets — become and Vault.

CHEATSHEET – DATA TO ROLES

A Precedence (low to high)

SOURCE	WINS?
role defaults	loses
group_vars to host_vars	host wins
play to task vars	task wins
extra-vars -e	always wins

B Logic and Files

NEED	USE
Branch	when: bare, no delimiters
Repeat	loop: + loop_control label
Per-host file	template + handler
Share	role to collection, FQCN

● OK – converged, no change ● CHANGED – converged, did change ❑ FAILED – did not converge

END ENGINEERING · VOL 2

VOL 3 – PRODUCTION ANSIBLE

Vault, quality gates and CI/CD: from tested role to rolling fleet.